GUÍA DE LABORATORIO 1

Pruebas Unitarias de una API NestJS

Proyecto: notes-api
Materia: Verificación y Validación de Software (INF780)
Nivel: Universitario | Duración: 4–5 horas
Herramientas: NestJS · Jest · TypeScript

2026

Introducción

Esta guía de laboratorio te llevará paso a paso a construir y testear una API REST de notas (notes-api) usando NestJS. Al finalizar, habrás implementado una API completa en memoria y escrito pruebas unitarias profesionales con Jest para el servicio y el controlador.

Objetivos de aprendizaje

- Comprender la estructura de un proyecto NestJS
- Implementar un CRUD completo con almacenamiento en memoria
- Escribir pruebas unitarias para Services usando el principio AAA
- Mockear dependencias de servicios al testear Controllers
- Alcanzar una cobertura de código igual o mayor al 80%

Prerrequisitos

- Node.js 18+ instalado
- npm o yarn disponible en la terminal
- Conocimientos básicos de TypeScript y programación orientada a objetos
- Editor de código (VS Code recomendado)

Estructura del proyecto

Árbol de archivos · src/

```
src/
  main.ts                    # Entry point, ValidationPipe global
  app.module.ts              # Módulo raíz, importa NotesModule
  notes/
    dto/
      create-note.dto.ts     # Validaciones con class-validator
      update-note.dto.ts     # Campos opcionales (@IsOptional)
    entities/
      note.entity.ts         # Interfaz/clase Note
    notes.controller.ts      # Endpoints REST
    notes.controller.spec.ts # Tests del controlador
    notes.service.ts         # Lógica de negocio + datos en memoria
    notes.service.spec.ts    # Tests del servicio
    notes.module.ts          # Módulo de notas
```

Parte 1 — Crear el Proyecto

Paso 1 Instalar NestJS CLI y crear la aplicación

```
Terminal
# Instalar el CLI globalmente
npm install -g @nestjs/cli

# Crear el proyecto (seleccionar npm cuando se pregunte)
nest new notes-api

# Entrar al directorio
cd notes-api
```

Nota El CLI genera automáticamente la configuración de TypeScript, Jest y ESLint/Prettier. No es necesario configurar nada extra para comenzar.

Paso 2 Instalar dependencias adicionales

```
Terminal
npm install uuid class-validator class-transformer
npm install --save-dev @types/uuid
```

Paquetes instalados:

- uuid — para generar IDs únicos a cada nota
- class-validator / class-transformer — para validar los DTOs

Paso 3 Generar módulo, controlador y servicio

```
Terminal
nest generate module notes
nest generate controller notes
nest generate service notes
```

Resultado NestJS crea automáticamente los archivos en src/notes/ e inyecta NotesModule en AppModule. Los archivos .spec.ts también se generan listos para ser llenados.

Parte 2 — Implementación de la API

2.1 Entidad Note

Crea el archivo src/notes/entities/note.entity.ts con la siguiente estructura:

src/notes/entities/note.entity.ts

```
export class Note {
  id: string;
  title: string;
  content: string;
  createdAt: Date;
  updatedAt: Date;
}
```

2.2 DTOs con validaciones

src/notes/dto/create-note.dto.ts

```
import { IsNotEmpty, IsString, MaxLength, MinLength } from 'class-validator';

export class CreateNoteDto {
  @IsNotEmpty({ message: 'El título no puede estar vacío' })
  @IsString({ message: 'El título debe ser un texto' })
  @MinLength(3, { message: 'El título debe tener al menos 3 caracteres' })
  @MaxLength(100, { message: 'El título no puede superar los 100 caracteres' })
  title: string;

  @IsNotEmpty({ message: 'El contenido no puede estar vacío' })
  @IsString({ message: 'El contenido debe ser un texto' })
  @MinLength(1, { message: 'El contenido debe tener al menos 1 caracter' })
  @MaxLength(5000, { message: 'El contenido no puede superar los 5000 caracteres'
})
  content: string;
}
```

src/notes/dto/update-note.dto.ts

```
import { IsOptional, IsString, MaxLength, MinLength } from 'class-validator';

export class UpdateNoteDto {
  @IsOptional()
  @IsString({ message: 'El título debe ser un texto' })
```

```
  @MinLength(3, { message: 'El título debe tener al menos 3 caracteres' })
  @MaxLength(100, { message: 'El título no puede superar los 100 caracteres' })
  title?: string;

  @IsOptional()
  @IsString({ message: 'El contenido debe ser un texto' })
  @MinLength(1, { message: 'El contenido debe tener al menos 1 caracter' })
  @MaxLength(5000, { message: 'El contenido no puede superar los 5000 caracteres'
})
  content?: string;
}
```

2.3 Servicio — Lógica de negocio

El servicio almacena las notas en un arreglo en memoria. Implementa los siguientes métodos en notes.service.ts:

src/notes/notes.service.ts

```
import { Injectable, NotFoundException } from '@nestjs/common';
import { v4 as uuidv4 } from 'uuid';
import { CreateNoteDto } from './dto/create-note.dto';
import { UpdateNoteDto } from './dto/update-note.dto';
import { Note } from './entities/note.entity';

@Injectable()
export class NotesService {
  private notes: Note[] = [];

  findAll(): Note[] {
    return this.notes;
  }

  findOne(id: string): Note {
    const note = this.notes.find((n) => n.id === id);
    if (!note) {
      throw new NotFoundException(`Nota con id "${id}" no encontrada`);
    }
    return note;
  }

  create(createNoteDto: CreateNoteDto): Note {
    const note: Note = {
      id: uuidv4(),
      title: createNoteDto.title,
      content: createNoteDto.content,
      createdAt: new Date(),
```

```
      updatedAt: new Date(),
    };
    this.notes.push(note);
    return note;
  }

  update(id: string, updateNoteDto: UpdateNoteDto): Note {
    const note = this.findOne(id);
    if (updateNoteDto.title !== undefined) {
      note.title = updateNoteDto.title;
    }
    if (updateNoteDto.content !== undefined) {
      note.content = updateNoteDto.content;
    }
    note.updatedAt = new Date();
    return note;
  }

  remove(id: string): void {
    const index = this.notes.findIndex((n) => n.id === id);
    if (index === -1) {
      throw new NotFoundException(`Nota con id "${id}" no encontrada`);
    }
    this.notes.splice(index, 1);
  }
}
```

2.4 Controlador — Endpoints REST

Mapea los métodos del servicio a los verbos HTTP correspondientes:

Método	Ruta	Acción	Descripción
GET	`/notes`	`findAll`	Retorna todas las notas
GET	`/notes/:id`	`findOne`	Busca por ID; 404 si no existe
POST	`/notes`	`create`	Crea nota; responde 201 Created
PUT	`/notes/:id`	`update`	Actualiza campos; 404 si no existe
DELETE	`/notes/:id`	`remove`	Elimina nota; responde 204 No Content

`src/notes/notes.controller.ts`

```
import { Note } from './entities/note.entity';
import { NotesService } from './notes.service';
```

```
@Controller('notes')
export class NotesController {
  constructor(private readonly notesService: NotesService) {}

  @Get()
  findAll(): Note[] {
    return this.notesService.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string): Note {
    return this.notesService.findOne(id);
  }

  @Post()
  @UsePipes(new ValidationPipe({ whitelist: true }))
  @HttpCode(HttpStatus.CREATED)
  create(@Body() createNoteDto: CreateNoteDto): Note {
    return this.notesService.create(createNoteDto);
  }

  @Put(':id')
  @UsePipes(new ValidationPipe({ whitelist: true }))
  update(@Param('id') id: string, @Body() updateNoteDto: UpdateNoteDto): Note {
    return this.notesService.update(id, updateNoteDto);
  }

  @Delete(':id')
  @HttpCode(HttpStatus.NO_CONTENT)
  remove(@Param('id') id: string): void {
    return this.notesService.remove(id);
  }
}
```

2.5 Configurar ValidationPipe en main.ts

src/main.ts

```
import { ValidationPipe } from '@nestjs/common';
import { NestFactory } from '@nestjs/core';
import { AppModule } from './app.module';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
```

```
  app.useGlobalPipes(
    new ValidationPipe({
      whitelist: true,
      forbidNonWhitelisted: true,
      transform: true,
    }),
  );

  await app.listen(3000);
  console.log('API de notas corriendo en http://localhost:3000');
}

bootstrap();
```

Parte 3 — Pruebas Unitarias con Jest

> **Recuerda** Un test unitario prueba UNA sola unidad de forma aislada. Eso significa que cuando probamos el controlador, mockeamos el servicio, y viceversa. No usamos base de datos real ni HTTP real.

3.1 Tests del Servicio (notes.service.spec.ts)

El servicio no tiene dependencias externas (solo usa un array en memoria), por lo que podemos probarlo directamente sin mocks.

src/notes/notes.service.spec.ts

```
import { NotFoundException } from '@nestjs/common';
import { Test, TestingModule } from '@nestjs/testing';
import { CreateNoteDto } from './dto/create-note.dto';
import { UpdateNoteDto } from './dto/update-note.dto';
import { NotesService } from './notes.service';

describe('NotesService', () => {
  let service: NotesService;

  beforeEach(async () => {
    const module: TestingModule = await Test.createTestingModule({
      providers: [NotesService],
    }).compile();

    service = module.get<NotesService>(NotesService);
  });

  it('debe estar definido', () => {
    expect(service).toBeDefined();
  });

  // ─── findAll ──────────────────────────────────────────────────────────

  describe('findAll', () => {
    it('debe retornar un arreglo vacío si no hay notas', () => {
      const result = service.findAll();
      expect(result).toEqual([]);
    });

    it('debe retornar todas las notas creadas', () => {
      service.create({ title: 'Nota 1', content: 'Contenido 1' });
```

```
      service.create({ title: 'Nota 2', content: 'Contenido 2' });

      const result = service.findAll();
      expect(result).toHaveLength(2);
      expect(result[0].title).toBe('Nota 1');
      expect(result[1].title).toBe('Nota 2');
    });
  });

  // ─── findOne ──────────────────────────────────────────────────────────

  describe('findOne', () => {
    it('debe retornar una nota por su id', () => {
      const created = service.create({ title: 'Mi nota', content: 'Mi contenido'
});
      const found = service.findOne(created.id);
      expect(found).toEqual(created);
    });

    it('debe lanzar NotFoundException si la nota no existe', () => {
      expect(() => service.findOne('id-inexistente')).toThrow(NotFoundException);
    });

    it('debe lanzar NotFoundException con el mensaje correcto', () => {
      expect(() => service.findOne('abc-123')).toThrow(
        'Nota con id "abc-123" no encontrada',
      );
    });
  });

  // ─── create ───────────────────────────────────────────────────────────

  describe('create', () => {
    it('debe crear una nota con los datos proporcionados', () => {
      const dto: CreateNoteDto = { title: 'Nueva nota', content: 'Contenido nuevo'
};
      const note = service.create(dto);

      expect(note.id).toBeDefined();
      expect(note.title).toBe(dto.title);
      expect(note.content).toBe(dto.content);
      expect(note.createdAt).toBeInstanceOf(Date);
      expect(note.updatedAt).toBeInstanceOf(Date);
    });

    it('debe asignar un id único a cada nota', () => {
      const nota1 = service.create({ title: 'Nota A', content: 'Contenido A' });
```

```
      const nota2 = service.create({ title: 'Nota B', content: 'Contenido B' });
      expect(nota1.id).not.toBe(nota2.id);
    });

    it('debe agregar la nota a la lista interna', () => {
      service.create({ title: 'Nota X', content: 'Contenido X' });
      expect(service.findAll()).toHaveLength(1);
    });
  });

  // ─── update ──────────────────────────────────────────────────────────

  describe('update', () => {
    it('debe actualizar el título de una nota', () => {
      const created = service.create({ title: 'Original', content: 'Contenido' });
      const updated = service.update(created.id, { title: 'Actualizado' });
      expect(updated.title).toBe('Actualizado');
      expect(updated.content).toBe('Contenido');
    });

    it('debe actualizar el contenido de una nota', () => {
      const created = service.create({ title: 'Título', content: 'Viejo contenido'
});
      const updated = service.update(created.id, { content: 'Nuevo contenido' });
      expect(updated.content).toBe('Nuevo contenido');
      expect(updated.title).toBe('Título');
    });

    it('debe actualizar título y contenido simultáneamente', () => {
      const created = service.create({ title: 'Título', content: 'Contenido' });
      const dto: UpdateNoteDto = { title: 'Nuevo título', content: 'Nuevo
contenido' };
      const updated = service.update(created.id, dto);
      expect(updated.title).toBe('Nuevo título');
      expect(updated.content).toBe('Nuevo contenido');
    });

    it('debe actualizar updatedAt al modificar la nota', () => {
      const created = service.create({ title: 'Título', content: 'Contenido' });
      const fechaOriginal = created.updatedAt;
      const updated = service.update(created.id, { title: 'Cambiado' });

expect(updated.updatedAt.getTime()).toBeGreaterThanOrEqual(fechaOriginal.getTime()
);
    });

    it('debe lanzar NotFoundException si la nota a actualizar no existe', () => {
```

```
      expect(() => service.update('id-inexistente', { title: 'X' })).toThrow(
        NotFoundException,
      );
    });
  });

  // ── remove ──────────────────────────────────────────────────────────────

  describe('remove', () => {
    it('debe eliminar una nota existente', () => {
      const created = service.create({ title: 'Para borrar', content: 'Contenido'
});
      service.remove(created.id);
      expect(service.findAll()).toHaveLength(0);
    });

    it('debe lanzar NotFoundException si la nota a eliminar no existe', () => {
      expect(() => service.remove('id-inexistente')).toThrow(NotFoundException);
    });

    it('debe lanzar NotFoundException con el mensaje correcto al eliminar', () =>
{
      expect(() => service.remove('xyz-789')).toThrow(
        'Nota con id "xyz-789" no encontrada',
      );
    });

    it('debe eliminar solo la nota indicada y mantener las demás', () => {
      const nota1 = service.create({ title: 'Nota 1', content: 'Contenido 1' });
      const nota2 = service.create({ title: 'Nota 2', content: 'Contenido 2' });

      service.remove(nota1.id);

      const notas = service.findAll();
      expect(notas).toHaveLength(1);
      expect(notas[0].id).toBe(nota2.id);
    });
  });
});
```

3.2 Tests del Controlador (notes.controller.spec.ts)

El controlador depende del servicio. Usamos un mock con jest.fn() para aislar el controlador y verificar que delega correctamente.

src/notes/notes.controller.spec.ts

```
import { NotFoundException } from '@nestjs/common';
import { Test, TestingModule } from '@nestjs/testing';
import { CreateNoteDto } from './dto/create-note.dto';
import { UpdateNoteDto } from './dto/update-note.dto';
import { Note } from './entities/note.entity';
import { NotesController } from './notes.controller';
import { NotesService } from './notes.service';

const mockNote = (overrides?: Partial<Note>): Note => ({
  id: 'uuid-1234',
  title: 'Nota de prueba',
  content: 'Contenido de prueba',
  createdAt: new Date('2024-01-01'),
  updatedAt: new Date('2024-01-01'),
  ...overrides,
});

const mockNotesService = {
  findAll: jest.fn(),
  findOne: jest.fn(),
  create: jest.fn(),
  update: jest.fn(),
  remove: jest.fn(),
};

describe('NotesController', () => {
  let controller: NotesController;
  let service: typeof mockNotesService;

  beforeEach(async () => {
    const module: TestingModule = await Test.createTestingModule({
      controllers: [NotesController],
      providers: [{ provide: NotesService, useValue: mockNotesService }],
    }).compile();

    controller = module.get<NotesController>(NotesController);
    service = module.get(NotesService);

    jest.clearAllMocks();
  });

  it('debe estar definido', () => {
    expect(controller).toBeDefined();
  });

  // ─── findAll ──────────────────────────────────────────────────────────
```

```
  describe('findAll', () => {
    it('debe llamar a service.findAll y retornar la lista de notas', () => {
      const notes = [mockNote(), mockNote({ id: 'uuid-5678', title: 'Otra nota'
})];
      service.findAll.mockReturnValue(notes);

      const result = controller.findAll();

      expect(service.findAll).toHaveBeenCalledTimes(1);
      expect(result).toEqual(notes);
    });

    it('debe retornar un arreglo vacío cuando no hay notas', () => {
      service.findAll.mockReturnValue([]);

      const result = controller.findAll();

      expect(result).toEqual([]);
    });
  });

  // ── findOne ──────────────────────────────────────────────────────────

  describe('findOne', () => {
    it('debe llamar a service.findOne con el id correcto y retornar la nota', ()
=> {
      const note = mockNote();
      service.findOne.mockReturnValue(note);

      const result = controller.findOne('uuid-1234');

      expect(service.findOne).toHaveBeenCalledWith('uuid-1234');
      expect(result).toEqual(note);
    });

    it('debe propagar NotFoundException cuando la nota no existe', () => {
      service.findOne.mockImplementation(() => {
        throw new NotFoundException('Nota con id "inexistente" no encontrada');
      });

      expect(() => controller.findOne('inexistente')).toThrow(NotFoundException);
    });
  });

  // ── create ───────────────────────────────────────────────────────────
```

```
  describe('create', () => {
    it('debe llamar a service.create con el DTO y retornar la nota creada', () =>
{
      const dto: CreateNoteDto = { title: 'Nueva nota', content: 'Nuevo contenido'
};
      const note = mockNote({ title: dto.title, content: dto.content });
      service.create.mockReturnValue(note);

      const result = controller.create(dto);

      expect(service.create).toHaveBeenCalledWith(dto);
      expect(result).toEqual(note);
    });

    it('debe retornar la nota con id y fechas generadas', () => {
      const dto: CreateNoteDto = { title: 'Nota con id', content: 'Contenido' };
      const note = mockNote();
      service.create.mockReturnValue(note);

      const result = controller.create(dto);

      expect(result.id).toBeDefined();
      expect(result.createdAt).toBeInstanceOf(Date);
    });
  });

  // ─── update ──────────────────────────────────────────────────────────────

  describe('update', () => {
    it('debe llamar a service.update con id y DTO correctos', () => {
      const dto: UpdateNoteDto = { title: 'Título actualizado' };
      const updatedNote = mockNote({ title: 'Título actualizado' });
      service.update.mockReturnValue(updatedNote);

      const result = controller.update('uuid-1234', dto);

      expect(service.update).toHaveBeenCalledWith('uuid-1234', dto);
      expect(result.title).toBe('Título actualizado');
    });

    it('debe propagar NotFoundException cuando la nota a actualizar no existe', ()
=> {
      service.update.mockImplementation(() => {
        throw new NotFoundException('Nota con id "inexistente" no encontrada');
      });

      expect(() => controller.update('inexistente', { title: 'X' })).toThrow(
```

```
        NotFoundException,
      );
    });
  });

  // ─── remove ──────────────────────────────────────────────────────────

  describe('remove', () => {
    it('debe llamar a service.remove con el id correcto', () => {
      service.remove.mockReturnValue(undefined);

      controller.remove('uuid-1234');

      expect(service.remove).toHaveBeenCalledWith('uuid-1234');
      expect(service.remove).toHaveBeenCalledTimes(1);
    });

    it('debe propagar NotFoundException cuando la nota a eliminar no existe', ()
=> {
      service.remove.mockImplementation(() => {
        throw new NotFoundException('Nota con id "inexistente" no encontrada');
      });

      expect(() => controller.remove('inexistente')).toThrow(NotFoundException);
    });
  });
});
```

Parte 4 — Ejecutar y Validar las Pruebas

Paso 4 **Correr los tests en modo watch**

```
Terminal
# Ejecutar todos los tests una vez
npm run test

# Modo watch (se re-ejecutan al guardar cambios)
npm run test:watch

# Generar reporte de cobertura
npm run test:cov
```

Interpretando el reporte de cobertura

Después de ejecutar npm run test:cov verás una tabla similar a esta:

```
Reporte de cobertura esperado
----------|---------|----------|---------|---------|
File      | % Stmts | % Branch | % Funcs | % Lines |
----------|---------|----------|---------|---------|
notes/    |         |          |         |         |
  service |   95.2  |   88.9   |   100   |  95.2   |
  control |   87.5  |   75.0   |   87.5  |  87.5   |
----------|---------|----------|---------|---------|
All files |   91.3  |   81.9   |   93.7  |  91.3   | ✅
```

Meta La cobertura mínima esperada para esta práctica es 80% en todas las columnas. Si estás por debajo, revisa qué ramas (Branch) no están siendo ejercidas.

Recursos de Referencia

- Documentación oficial NestJS Testing: https://docs.nestjs.com/fundamentals/testing
- Jest API Reference: https://jestjs.io/docs/api
- class-validator decoradores: https://github.com/typestack/class-validator
- uuid package: https://www.npmjs.com/package/uuid